



ALGORYTMY I STRUKTURY DANYCH

Termin lab.: piątek 13:30

Kierunek: Informatyka

Grupa lab.: 5

SPRAWOZDANIE Z LABORATORIUM

4. Algorytmy z powracaniem

Autorzy:

PRO100KatYT

Burdziq

Prowadzący zajęcia:

mgr inż. Domomod

Rok akademicki 2025/2026

Spis treści

1	O programie	1
1.1	Opis i uzasadnienie wyboru reprezentacji	1
1.2	Prezentacja działania programu	1
2	Wizualizacja wygenerowanych grafów	2
3	Wykresy benchmarków i analiza wyników	3
4	Obserwacje związane z algorytmami	4
5	Podsumowanie	4

1 O programie

1.1 Opis i uzasadnienie wyboru reprezentacji

Program służy do generowania grafów hamiltonowskich o zadanym nasyceniu oraz niehamiltonowskich o nasyceniu 50%. Głównym celem programu jest znajdowanie cyklu Hamiltona za pomocą algorytmu Robertsa-Floresa oraz cyklu Eulera. Dodatkowo program umożliwia wypisanie grafu oraz jego eksport do formatu TikZ. Użyliśmy macierzy sąsiedztwa, ponieważ gwarantuje ona sprawdzenie istnienia krawędzi grafu w czasie $O(1)$, co jest kluczowe dla wydajności algorytmu z powracaniem Robertsa-Floresa, a przy liczbie 50 wierzchołków (ta liczba jest capped w programie) jej narzut pamięciowy jest całkowicie pomijalny w stosunku do zysków czasowych względem listy krawędzi czy następników.

1.2 Prezentacja działania programu

Poniżej przedstawiono przykłady działania programu w trybach `hamilton` oraz `non-hamilton`.

Tryb Hamilton

```
Terminal

$ python3 main.py --hamilton
nodes> 13
saturation> 42.0
action> print
| 1 2 3 4 5 6 7 8 9 10 11 12 13
-----
1 | 0 1 1 0 1 1 1 0 0 1 0 1 1
2 | 1 0 1 1 0 1 0 1 0 0 0 1 0
3 | 1 1 0 0 1 1 0 0 0 0 1 1 0
4 | 0 1 0 0 0 1 1 0 1 1 1 0 0
5 | 1 0 1 0 0 0 1 0 1 0 0 0 0
6 | 1 1 1 1 0 0 1 0 0 0 0 0 1
7 | 1 0 0 1 1 1 0 0 0 0 0 0 0
8 | 0 1 0 0 0 0 0 0 0 1 0 1 1
9 | 0 0 0 1 1 0 0 0 0 1 1 0 0
10 | 1 0 0 1 0 0 0 1 1 0 0 1 1
11 | 0 0 1 1 0 0 0 0 1 0 0 1 0
12 | 1 1 1 0 0 0 0 1 0 1 1 0 0
13 | 1 0 0 0 0 1 0 1 0 1 0 0 0
action> hamilton
Searching... Found!
1, 13, 10, 12, 8, 2, 6, 7, 5, 9, 4, 11, 3, 1
action> euler
Searching... Found!
1, 2, 3, 1, 5, 3, 6, 1, 7, 4, 2, 6, 4, 9, 5, 7, 6,
13, 1, 10, 4, 11, 3, 12, 2, 8, 10, 9, 11, 12,
8, 13, 10, 12, 1
action> exit
Program exited with status: 0
```

Tryb Non-Hamilton

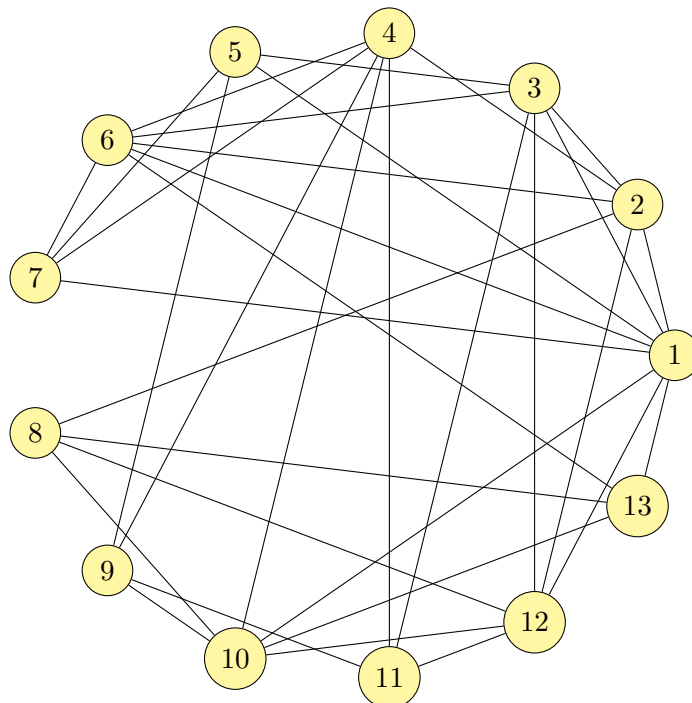
```
Terminal

$ python3 main.py --non-hamilton
nodes> 13
action> print
| 1 2 3 4 5 6 7 8 9 10 11 12 13
-----
1 | 0 0 0 0 0 0 0 0 0 0 0 0 0
2 | 0 0 1 0 0 0 0 1 1 0 1 1 1
3 | 0 1 0 1 0 1 0 1 1 0 0 0 0
4 | 0 0 1 0 0 1 0 1 1 0 1 0 0
5 | 0 0 0 0 0 1 0 1 1 0 0 0 0
6 | 0 0 1 1 1 0 0 1 1 1 1 0 1
7 | 0 0 0 0 0 0 0 0 1 1 1 1 0
8 | 0 1 1 1 1 1 0 0 0 0 0 1 0
9 | 0 1 1 1 1 1 1 0 0 1 1 0 1
10 | 0 0 0 0 0 1 1 0 1 0 1 1 1
11 | 0 1 0 1 0 1 1 1 0 1 1 0 0
12 | 0 1 0 0 0 0 1 1 0 1 0 0 0
13 | 0 1 0 0 0 1 0 0 1 1 1 0 0
action> hamilton
Searching...
There is no Hamilton cycle :(
action> euler
Searching... Found!
2, 3, 4, 6, 3, 8, 2, 9, 4, 8, 5, 6, 8, 12, 2, 11, 6,
9, 7, 10, 6, 13, 9, 10, 11, 7, 12, 10, 13,
11, 9, 2, 5, 4, 3
action> exit
Program exited with status: 0
```

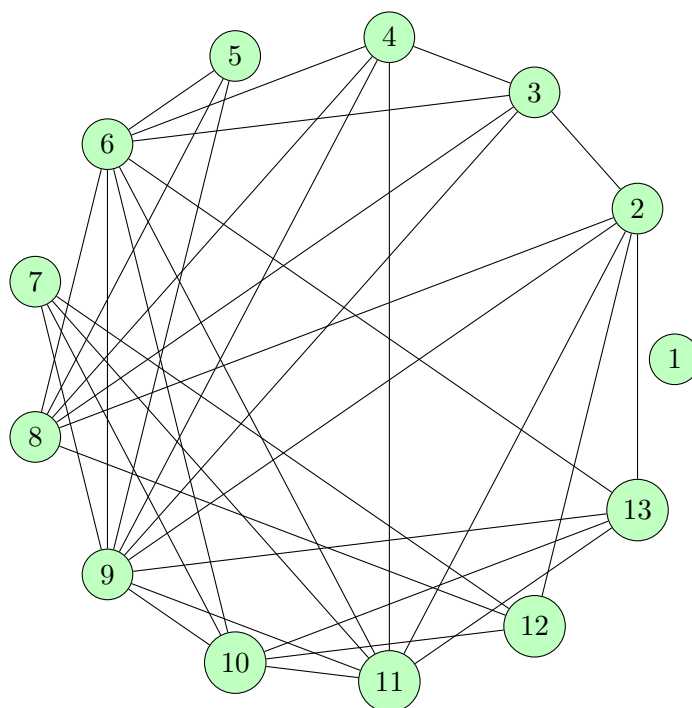
Rysunek 1: Działanie programu.

2 Wizualizacja wygenerowanych grafów

Poniżej znajdują się wizualizacje grafów wygenerowanych w poprzednim punkcie i wyeksportowanych z programu do formatu TikZ.



Rysunek 2: Graf wygenerowany w trybie Hamilton o saturacji 42%.

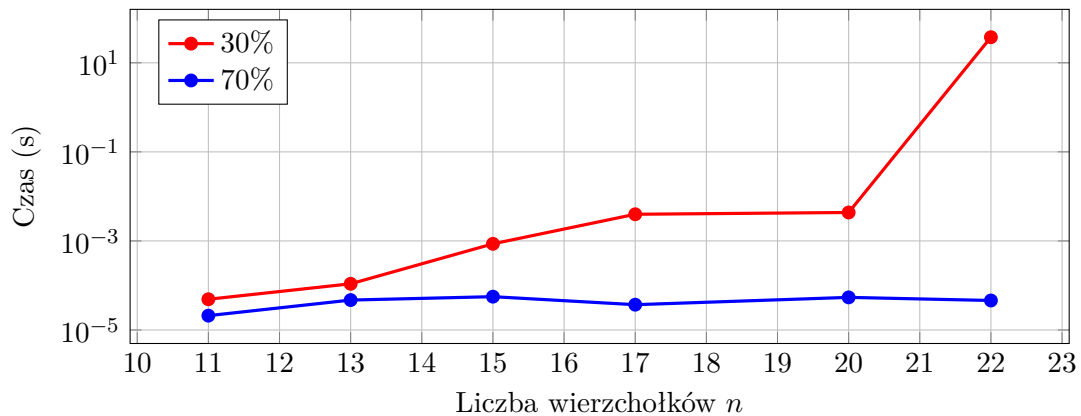


Rysunek 3: Graf wygenerowany w trybie Non-Hamilton.

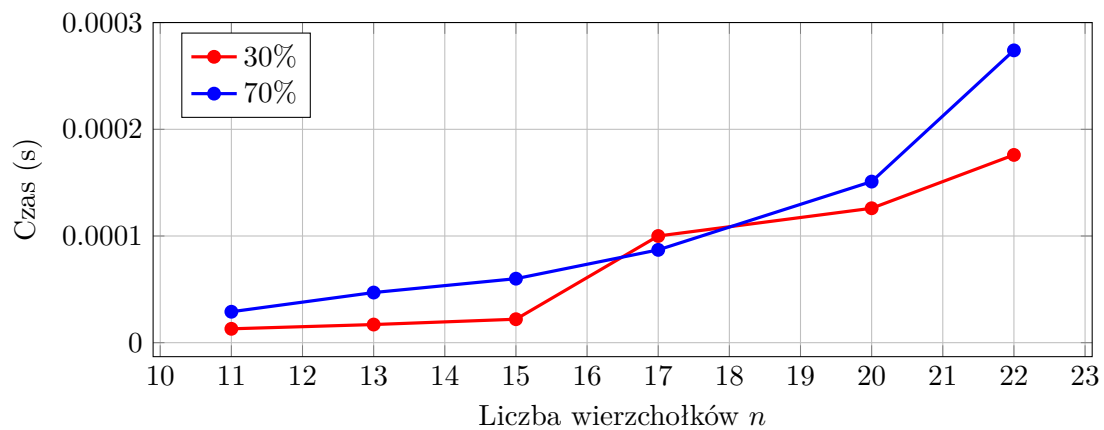
3 Wykresy benchmarków i analiza wyników

Poniższe wykresy prezentują złożoność obliczeniową algorytmów Hamiltona i Eulera. Na osiach rzędnych wykresów Hamiltona zastosowano skalę logarytmiczną, aby uwidocznić ogromne różnice w czasie wykonania (od milisekund do ponad 2 godzin w trybie Non-Hamilton).

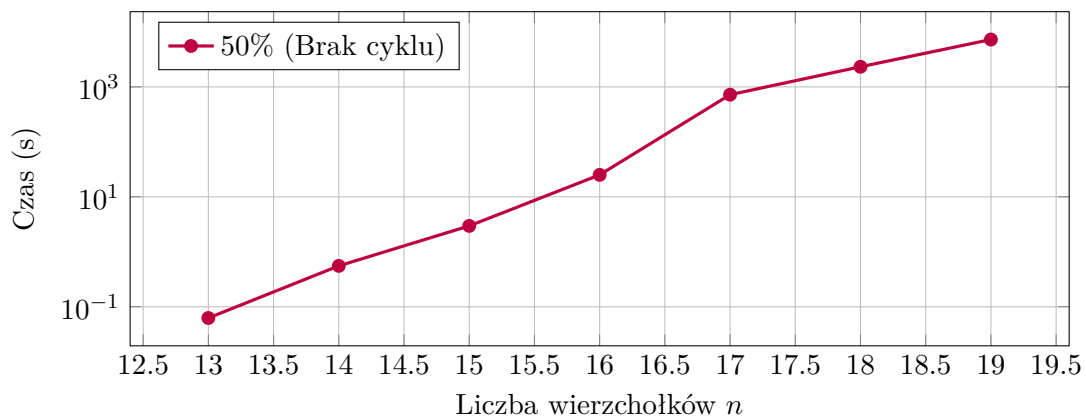
Cykl Hamiltona (Grafy Hamiltonowskie)



Cykl Eulera (Grafy Hamiltonowskie)



Cykl Hamiltona (Grafy Nie-Hamiltonowskie 50%)



4 Obserwacje związane z algorytmami

Wykresy dla grafów hamiltonowskich prezentują przebieg tego przeszukiwania dla nasycenia 30% i 70%, natomiast dla grafów niehamiltonowskich – 50%. Dla pierwszego wykresu czas poszukiwania dla 30% nasycenia jest dla każdej liczby wierzchołków większy niż czas poszukiwania dla 70% nasycenia. Dla grafów o nasyceniu 70% czas ten waha się pomiędzy 10^{-5} a 10^{-4} sekundy, natomiast dla grafów o nasyceniu 30% dla $n \leq 20$ czas rośnie powoli, maksymalnie osiągając wartość w okolicach 10^{-2} sekundy, natomiast dla $n > 20$ czas gwałtownie wzrasta do ponad 10 sekund.

W przypadku poszukiwania cyklu Eulera sprawa ma się trochę inaczej. Dla 30% grafów tym razem wartości są zdecydowanie mniejsze i dla liczby wierzchołków z zakresu $\langle 11; 22 \rangle$ nie przekraczają $2 \cdot 10^{-4}$ sekundy. W tym wypadku grafy o nasyceniu 70% przeszukiwane są trochę wolniej, jednak ich czas znacząco nie odbiega od czasu dla grafów 30%, ale mimo wszystko będąc jednak nieznacznie wyższy.

Dla poszukiwania cyklu Hamiltona w grafie niehamiltonowskim o nasyceniu 50% czas mieści się w zdecydowanie innych przedziałach. Do okolic 16 wierzchołków czas ten jest w miarę akceptowalny i nieznacznie przekracza 10 sekund. Dla $n \geq 17$ czas ten jednak wynosi już ponad 1000 sekund, czyli można go liczyć już w minutach, a dla $n \geq 20$ nawet w godzinach. Mimo takiego czasu przeszukiwania algorytm i tak nie znajdzie cyklu Hamiltona w grafie niehamiltonowskim bo jest to po prostu niemożliwe.

5 Podsumowanie

W ramach zadania zaimplementowaliśmy dwa algorytmy przeszukiwania grafu: algorytm Roberta-Floresa do poszukiwania cyklu Hamiltona oraz algorytm poszukiwania cyklu Eulera w grafie. Do przechowywania danych na temat wierzchołków wykorzystaliśmy reprezentację grafu w postaci macierzy sąsiedztwa. Wygenerowaliśmy również grafy hamiltonowskie o nasyceniu 30% i 70% oraz grafy niehamiltonowskie o nasyceniu 50%, każdy zawierający n wierzchołków z przedziału $\langle 11; 22 \rangle$. Na wykresach przedstawiliśmy czas przeszukiwania każdego grafu o n wierzchołkach podanymi powyżej algorytmami. Dzięki temu projektowi dowiedzieliśmy się jak generować grafy hamiltonowskie i niehamiltonowskie oraz poznaliśmy algorytmy przeszukiwania grafów w celu znalezienia cyklu Hamiltona i Eulera.